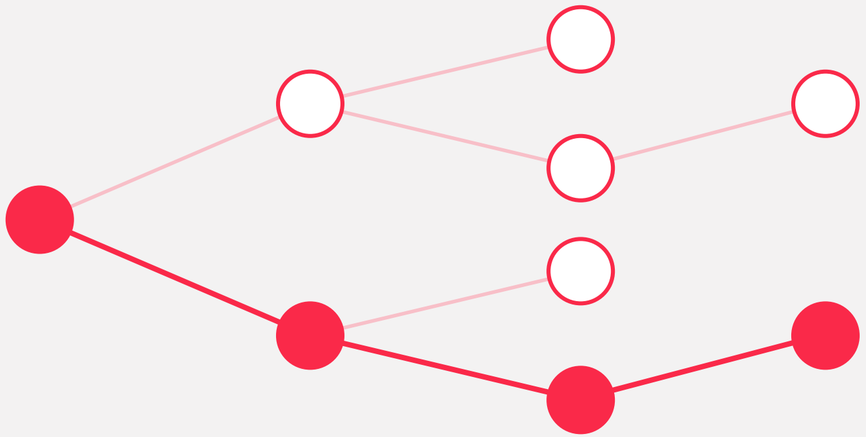
Búsqueda en Espacios de Estados

Unidad 2 — Algoritmos de búsqueda no informada, informada y adversarial.



Contenido

01 **Representación formal de un espacio de estados** — Diapositivas 4–5

02 **Búsqueda no informada: BFS y DFS** — Diapositivas 7–10

03 **Búsqueda informada: A* y Búsqueda Voraz** — Diapositivas 12–15

04 **Comparación de algoritmos** — Diapositiva 17

05 **Búsqueda adversarial: Minimax (Tres en Raya)** — Diapositivas 19–20

01

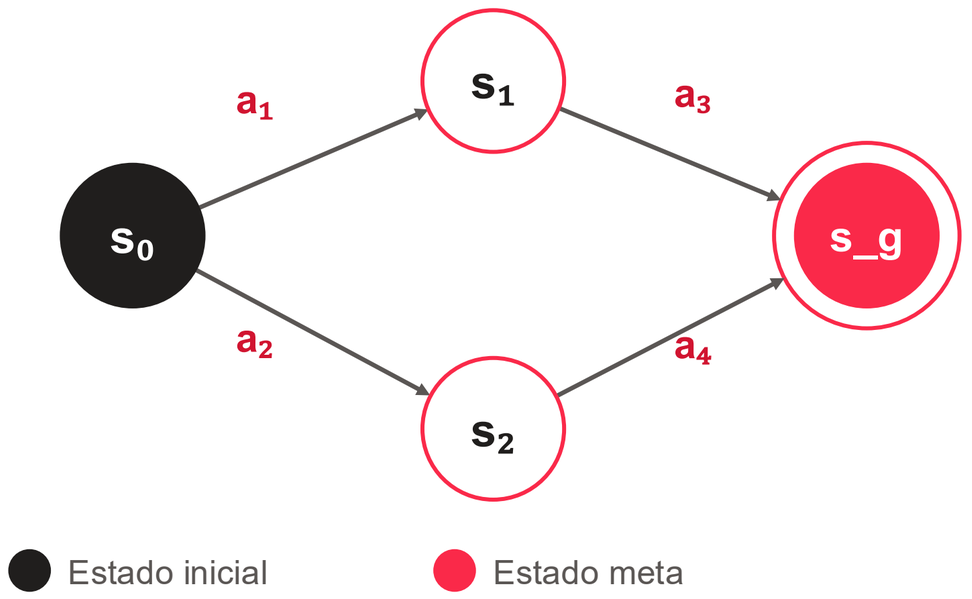
Representación Formal de un Espacio de Estados

01

Un espacio de estados como grafo dirigido

Los nodos representan los estados; las aristas, las acciones que permiten pasar de un estado a otro.

Grafo del espacio de estados



$S = \{s_0, s_1, \ldots, s_n\}$

Conjunto de estados

$O = \{a_1, a_2, \ldots, a_m\}$

Conjunto de operadores (acciones)

$f(s, a) \rightarrow s'$

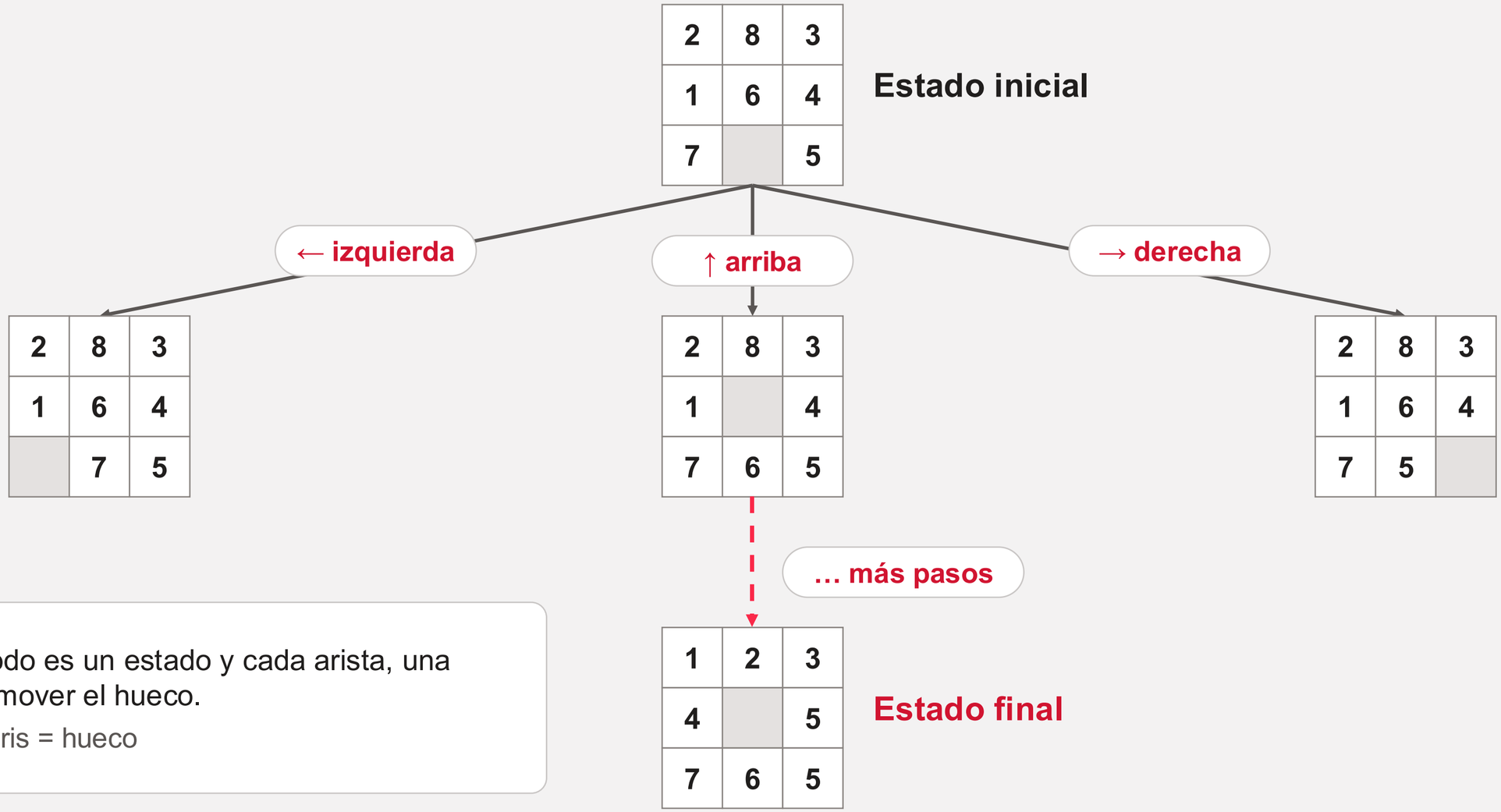
Función de transición: aplicar a en s produce s′

$S_goal \subset S$

Estado(s) meta

El 8-puzzle es el problema usado en toda la unidad para poner en práctica esta representación.

El 8-puzzle como espacio de estados



Cada nodo es un estado y cada arista, una acción: mover el hueco.

Casilla gris = hueco

02

Búsqueda No Informada: BFS y DFS

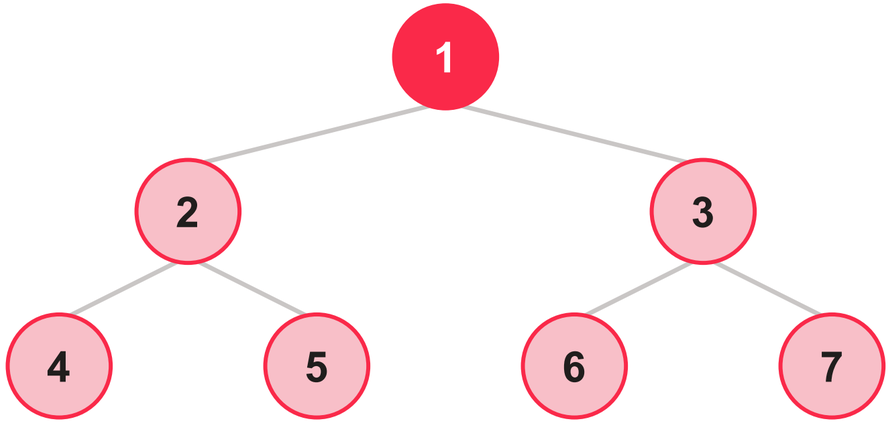
02

¿Qué es la búsqueda no informada?

Explora el espacio de estados sin información sobre qué tan cerca está la solución: solo conoce el estado inicial, el estado meta y las transiciones posibles.

BFS · Breadth-First

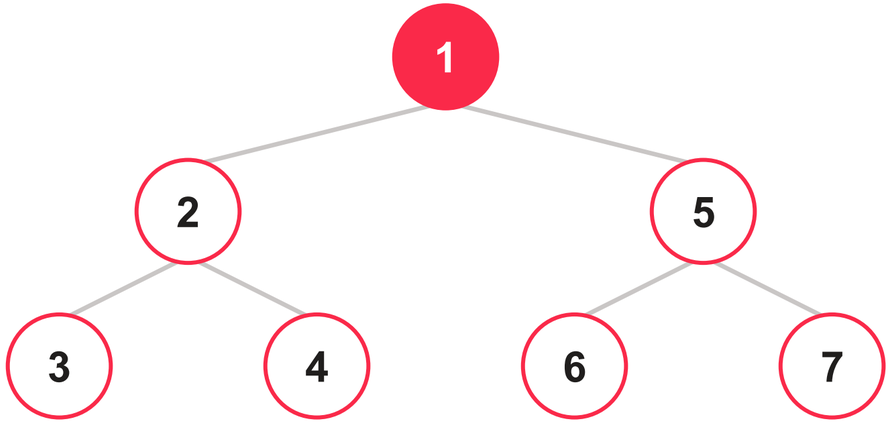
Explora por niveles, de lado a lado



Cola (FIFO): sale el primero que entró

DFS · Depth-First

Baja por un camino hasta el final y retrocede



Pila (LIFO): sale el último que entró

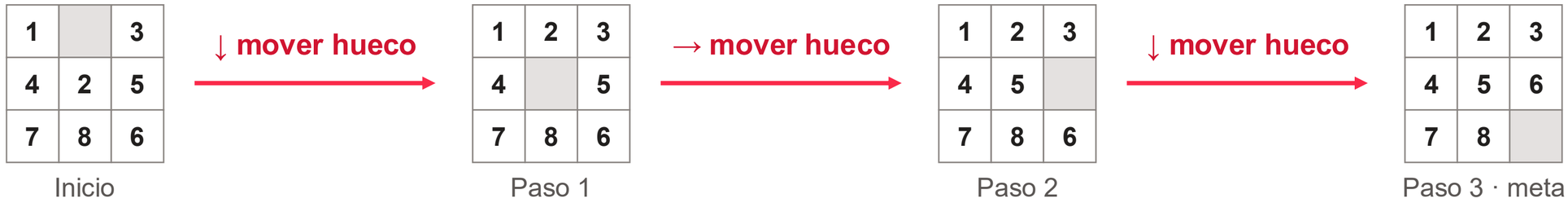
BFS resuelve el 8-puzzle

`estado_inicial = "103425786"`

Resultado real: solución en 3 pasos

BFS explora nivel por nivel, así que la primera solución que encuentra ya es la más corta u óptima posible.

```
def bfs_8_puzzle(inicial, meta):
    cola = deque([inicial])
    predecesor = {inicial: None}
    while cola:
        actual = cola.popleft()   # FIFO
        for sucesor in generar_sucesores(actual):
            if sucesor not in predecesor:
                predecesor[sucesor] = actual
                cola.append(sucesor)
                if sucesor == meta:
                    return reconstruir_camino(predecesor, meta)
```



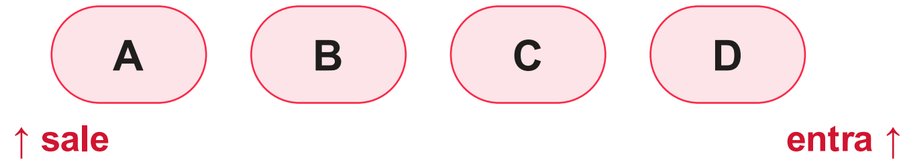
DFS resuelve el mismo puzzle

Mismo estado inicial y meta que con BFS.

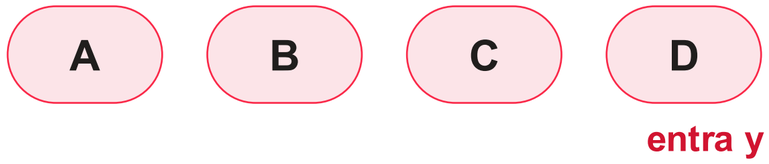
DFS solo cambia FIFO por LIFO pero encuentra un camino mucho más largo que el óptimo.

```python
def bfs_8_puzzle(inicial, meta):
    pila = [inicial]
    predecesor = {inicial: None}
    while pila:
        actual = pila.pop()   # LIFO
        for sucesor in generar_sucesores(actual):
            if sucesor not in predecesor:
                predecesor[sucesor] = actual
                pila.append(sucesor)
                if sucesor == meta:
                    return reconstruir_camino(predecesor, meta)
```

BFS · cola (FIFO)



DFS · pila (LIFO)



Mismo estado, mismo meta — ¿cuántos pasos?

3

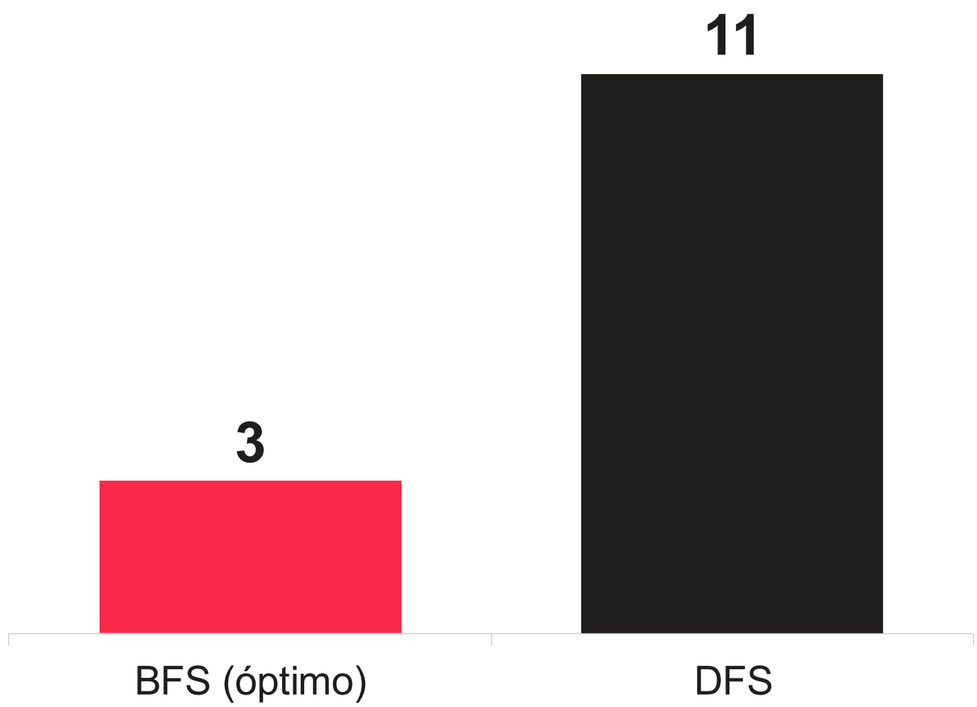
pasos — BFS (óptimo)

11

pasos — DFS

DFS encuentra una solución válida, pero al no explorar por niveles no hay garantía de que sea la más corta.

Longitud de la solución (pasos)



03

Búsqueda Informada: A* y Búsqueda Voraz

03

Heurísticas y la función f(n) = g(n) + h(n)

Función de evaluación

f(n) = g(n) + h(n)

g(n) costo real recorrido + **h(n)** estimación a la meta = **f(n)** prioridad en la frontera

Distancia de Manhattan (8-puzzle)

2	8	3
1	6	4
7		5

h(n) = 1 + 2 + 1 + 1 = 5

Fichas rosadas: fuera de su posición meta.

Heurística

Estima qué tan cerca está un estado de la meta, sin evaluar todas las posibilidades.

Manhattan

Suma de las distancias de cada ficha a su posición objetivo en el tablero.

heapq

Implementa la frontera como cola de prioridad: extrae siempre el estado con menor f(n).

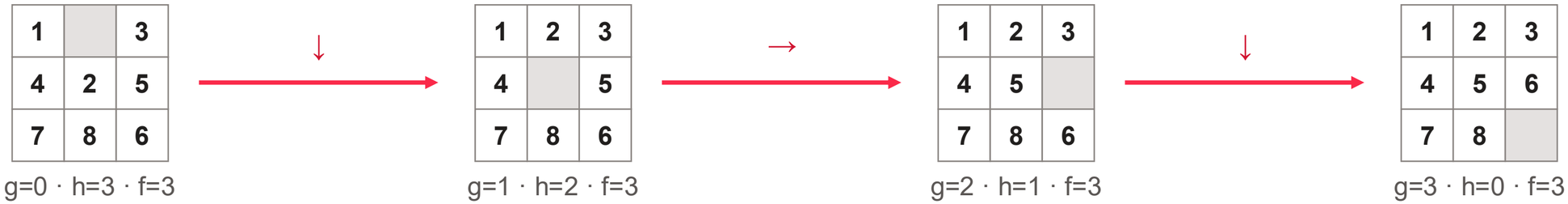
A* resuelve el mismo puzzle: 3 pasos

Sobre el mismo estado inicial de BFS/DFS, A* también encuentra la solución óptima: 3 pasos.

A diferencia de BFS, no explora un nivel completo: la heurística lo guía hacia estados prometedores.

```python
def heuristica_manhattan(estado, meta):
    distancia = 0
    for i, ficha in enumerate(estado):
        if ficha != "0":
            pos_meta = meta.index(ficha)
            fila_a, col_a = divmod(i, 3)
            fila_m, col_m = divmod(pos_meta, 3)
            distancia += abs(fila_a-fila_m) + abs(col_a-col_m)
    return distancia   # f(n) = g(n) + h(n)

# heapq mantiene el estado de menor f(n) al frente
```



Búsqueda Voraz: solo h(n)

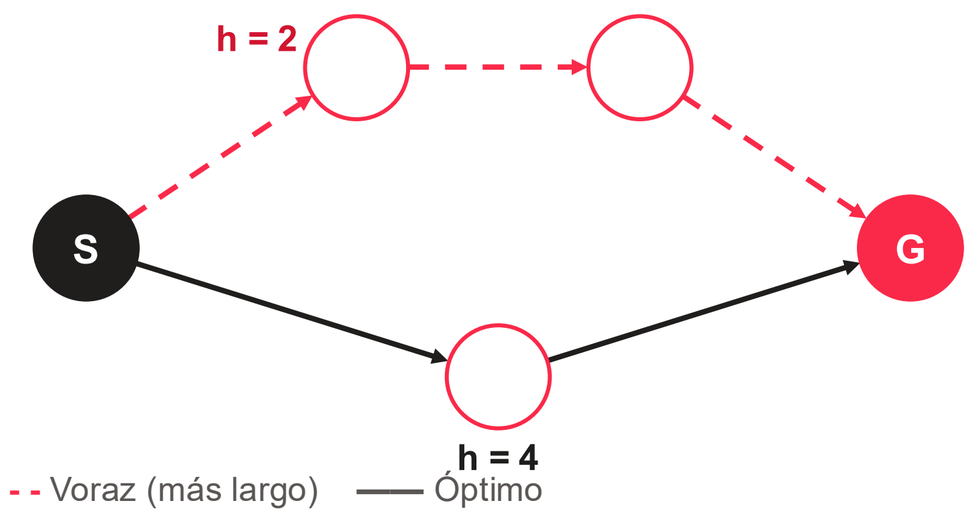
A* · usa g(n) y h(n)

f(n) = g(n) + h(n)

Voraz · ignora el costo g(n)

f(n) = h(n)

Por qué puede fallar (esquema ilustrativo)



Ventaja

Puede expandir menos nodos que A* cuando la heurística es muy precisa. En cada paso elige el estado que “parece” más cercano a la meta.

Limitación

No garantiza una solución óptima: puede comprometerse con un camino que resulte más largo, sin importar cuánto costó llegar.

A* vs. Voraz, en un caso donde sí se nota

8

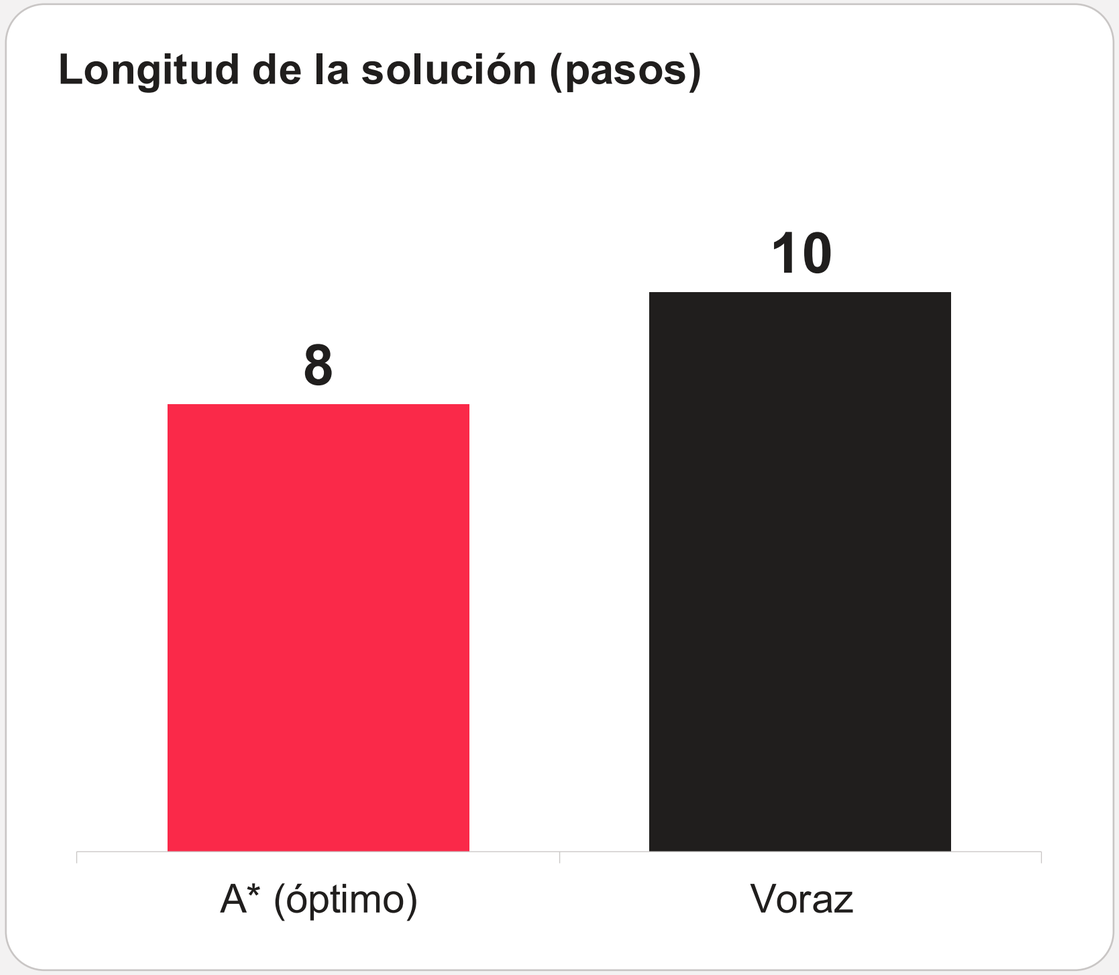
pasos — A* (óptimo)

10

pasos — Voraz (válida, no óptima)

Al guiarse solo por la heurística, Voraz avanzó por un camino que “parecía” prometedor, sin considerar la distancia ya recorrida.

A* combina g(n) y h(n), por eso encuentra el camino más corto.



04

Comparación de Algoritmos

04

BFS, DFS, Voraz y A*

Algoritmo	Frontera	Usa heurística	Óptimo	Memoria
BFS	FIFO (cola)	No	**Sí**	Alta
DFS	LIFO (pila)	No	No	Baja
Voraz	Prioridad, f(n) = h(n)	Sí	No	Moderada
A*	Prioridad, f(n) = g(n) + h(n)	Sí	**Sí (heurística admisible)**	Moderada

BFS garantiza el óptimo, pero su costo de memoria lo vuelve poco práctico en problemas grandes.

A* logra el mismo óptimo que BFS, guiado por la heurística, de forma mucho más eficiente.

05

Búsqueda Adversarial: Minimax (Tres en Raya)

05

El algoritmo Minimax

MAX (la IA)

Elige el movimiento que maximiza su puntaje.

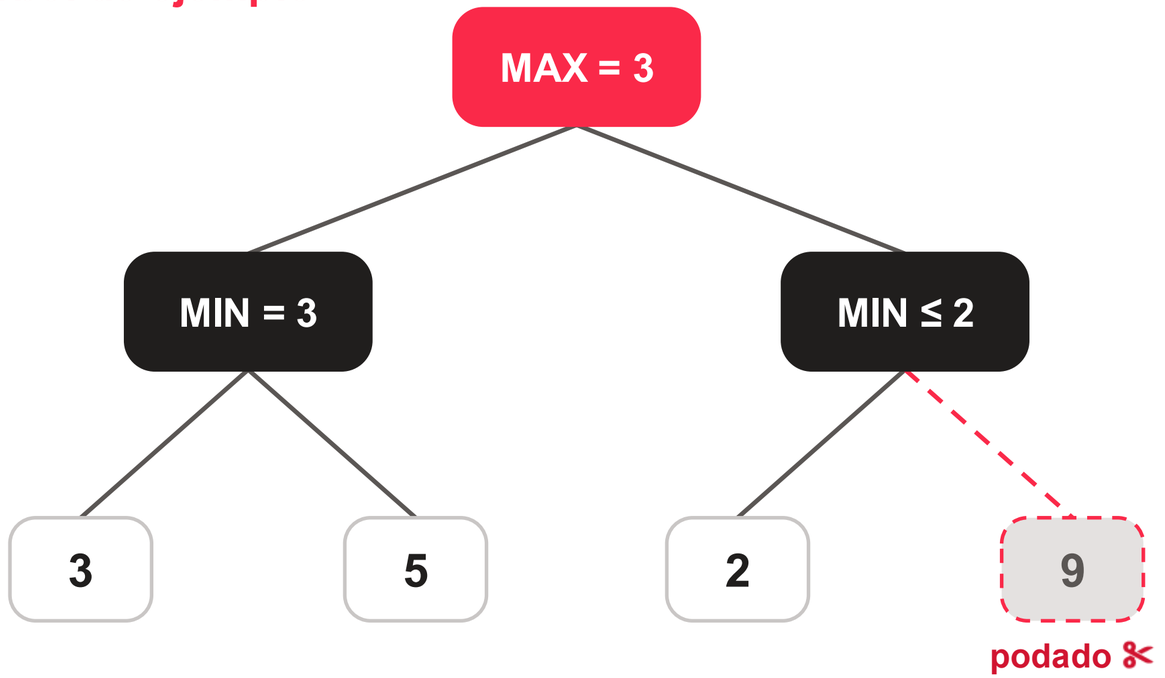
MIN (el oponente)

Elige el movimiento que minimiza el puntaje de la IA.

Puntaje terminal

+10 − profundidad si gana la IA · −10 + profundidad si gana el humano · 0 si hay empate.

Árbol de ejemplo



Poda alfa-beta: descarta ramas que ya no pueden cambiar la decisión final — mismo resultado, menos trabajo. Aquí MIN ya vale ≤ 2 < 3, así que no se mira la hoja 9.

La aplicación: minimax - tictactoe

●○○

Aleatoria total

La IA elige un movimiento al azar.

●●○

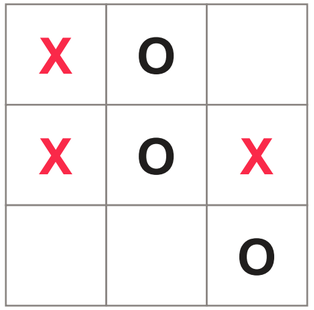
Fácil (subóptima)

Elige entre los movimientos razonablemente buenos.

●●●

Perfecta (MinMax completo)

Nunca pierde: juega el movimiento óptimo exacto en cada turno.



La aplicación también visualiza el árbol de búsqueda y qué ramas fueron podadas por alfa-beta.

Interfaz interactiva en Streamlit

```
$ cd Unidad 2/minimax-tictactoe
$ streamlit run app.py
```

RECURSOS

Resumen de la Unidad 2

	Tema	Recurso del curso
01	**Representación de estados, BFS, DFS, A*, Voraz**	Unidad 2/1_buscar-estados-ia.ipynb
02	**Búsqueda adversarial: Minimax + poda alfa-beta**	Unidad 2/minimax-tictactoe/ (app Streamlit)

BFS y A*

Garantizan la solución óptima; A* lo hace de forma mucho más eficiente gracias a la heurística.

DFS y Voraz

Sacrifican la garantía de optimalidad a cambio de menor costo de memoria o de cómputo.

¡Gracias!

Estos algoritmos de búsqueda son la base conceptual para todo el razonamiento por pasos que viene después en el curso.

